

Sparse-sensing state estimation of vortex-gust airfoil interactions

Toward model-based flow control

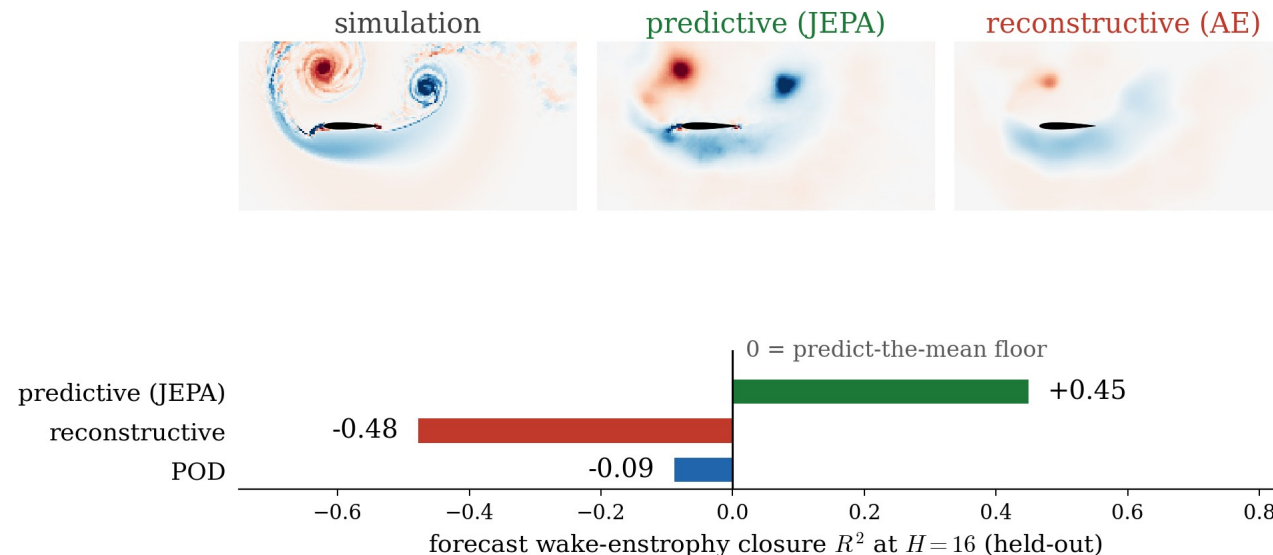
Forward physical closure of predictive vs reconstructive latents at $Re = 5000$

A. Solera-Rico, A. Miró, O. Lehmkuhl, C. Sanmiguel Vila



The control problem

- Small and micro air vehicles increasingly fly where the **gust speed is comparable to or above the flight speed** (urban canyons, building and ship wakes): gust ratio $G > 1$ brings large, fast load transients classical models miss
- Active control, model-based or RL, needs a **fast, faithful forward model** of the flow: a learned reduced dynamics it can trust under rollout
- This talk: **what reduced state is worth planning against?**

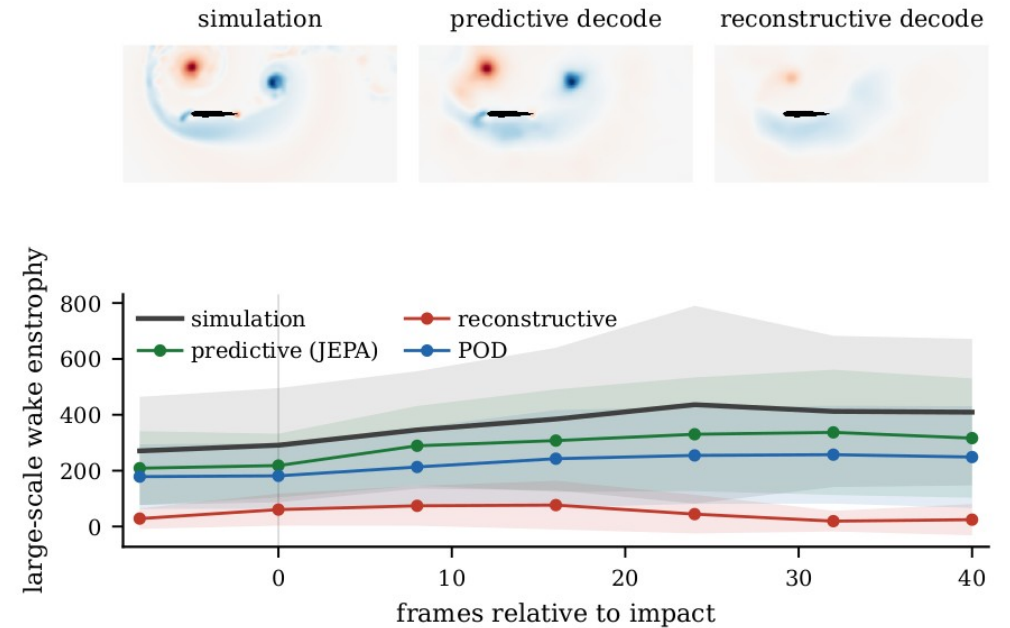


Top: encode-then-decode reconstruction of the field 16 frames after impact (case $G = -3$, $d = 64$); the predictive latent keeps the wake. Bars: held-out test_b forecast closure (predictor rollout, $H = 16$).

Sparse-sensing state estimation, vortex-gust interactions · EUROMECH colloquium

Why it is hard: it is the wake, not just the forces

- The load transient is built by the **leading-edge vortex (LEV)** and the wake it leaves; the discriminating information is in the **wake**, not the integrated forces C_L , C_D
- **Gaussian scale split** (Motoori & Goto 2019): low-pass the vorticity at $\sigma/c = 0.05$ into a large-scale part u_L (the LEV and shear layer that carry the lift) and a fine part u_S , then track the **large-scale wake enstrophy** through the encounter to see which reduced state keeps that load-bearing structure
- Top: large-scale vorticity at impact+16 for the **strongest test gust** ($G = -3$, $D = 1.5$, $Y = -0.1$): simulation vs the predictive and reconstructive **reconstructions** (defined shortly); predictive keeps the LEV/wake, reconstructive smooths it away
- Bands = **mean $\pm$ 1 s.d. across the test_b encounters** (encounter spread, not a confidence interval)



Large-scale ($\sigma/c = 0.05$) wake vorticity at impact+16 (top) and large-scale wake enstrophy vs frame (bottom), by family.

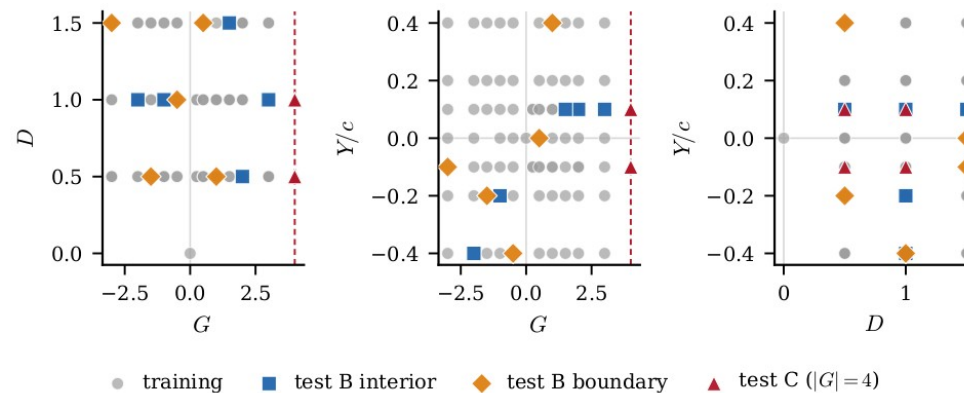
The opening for JEPA

- Most flow ROMs, POD/DMD, autoencoders, are trained to **reconstruct the field**
 - reconstruction fixes the latent only up to a diffeomorphism, so its geometry is not constrained to be predictable
- Alternative: learn a state that is **predictive by construction**, and plan against it

Which reduced state stays physically closed when propagated forward, so that every observable remains recoverable along the rollout?

Data

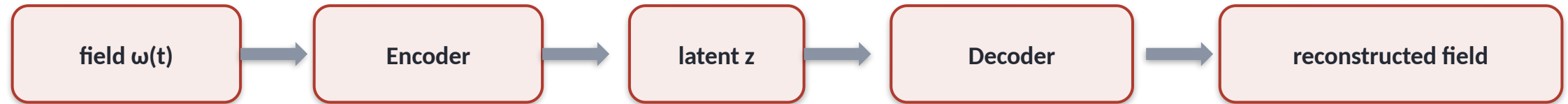
- **DNS** (SOD2D, no subgrid-scale model), NACA 0012, $\alpha = 14^\circ$, **Re = 5000**
- Taylor-vortex gusts parametrised by strength **G**, core diameter **D**, wall-normal offset **Y**
- 84 cases; impact-centred encounters; six observables: C_L , C_D , impulse I_y , **wake enstrophy**, $\pm$ circulation
- Held-out **test_b** (interpolation) and **test_c** ($|G| = 4$ extrapolation)



Training envelope in (G, D, Y) ; held-out interpolation and extrapolation.

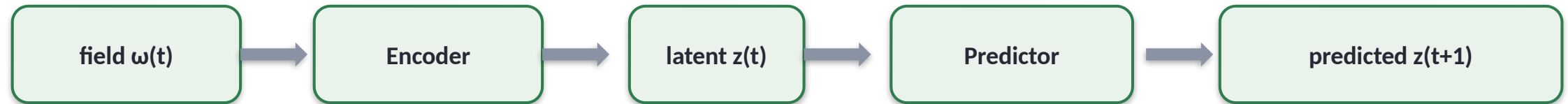
What is a JEPA? (1 / 2)

Autoencoder (most flow ROMs): reconstruct the field



loss compares the reconstruction to the input in PIXEL space

JEPA: predict the next latent, no decoder



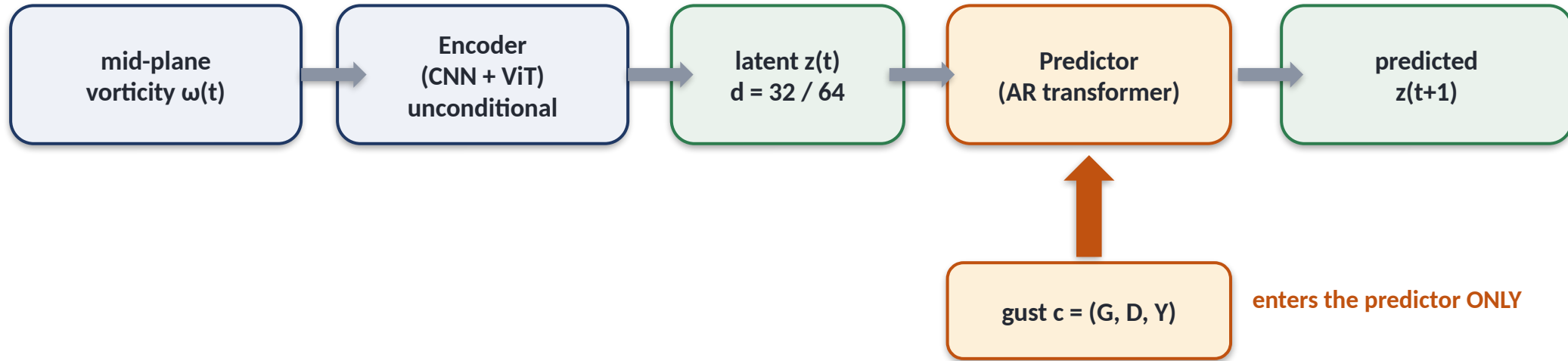
loss compares predicted $z(t+1)$ to $z(t+1) = \text{Encoder}(\text{next frame})$ in LATENT space; an anti-collapse term replaces the decoder

Reconstruction ties the latent to pixels; prediction makes it forecastable, which is what a controller needs.

What is a JEPA? (2 / 2): the recipe and why for control

- **Training, in one line:** minimise the **latent** prediction error plus an **anti-collapse term** so the encoder cannot cheat by mapping everything to a constant (no decoder, no pixel loss)
- So the encoder **keeps only what predicts the future** (the control-relevant dynamics), and the predictor **gets no pixel-level shortcuts**
- **Cheap to roll out:** planning happens in the $d = 32-64$ latent, two to three orders of magnitude cheaper than evolving the full field
- Lineage: I-JEPA / V-JEPA (image, video); from-pixels LeWM, LeJEPA, PLDM, so far only gridworld and toy visual tasks
- **Why for control:** a learned world-model that is **observable from sensors**; to our knowledge the **first end-to-end JEPA on a parametric flow**

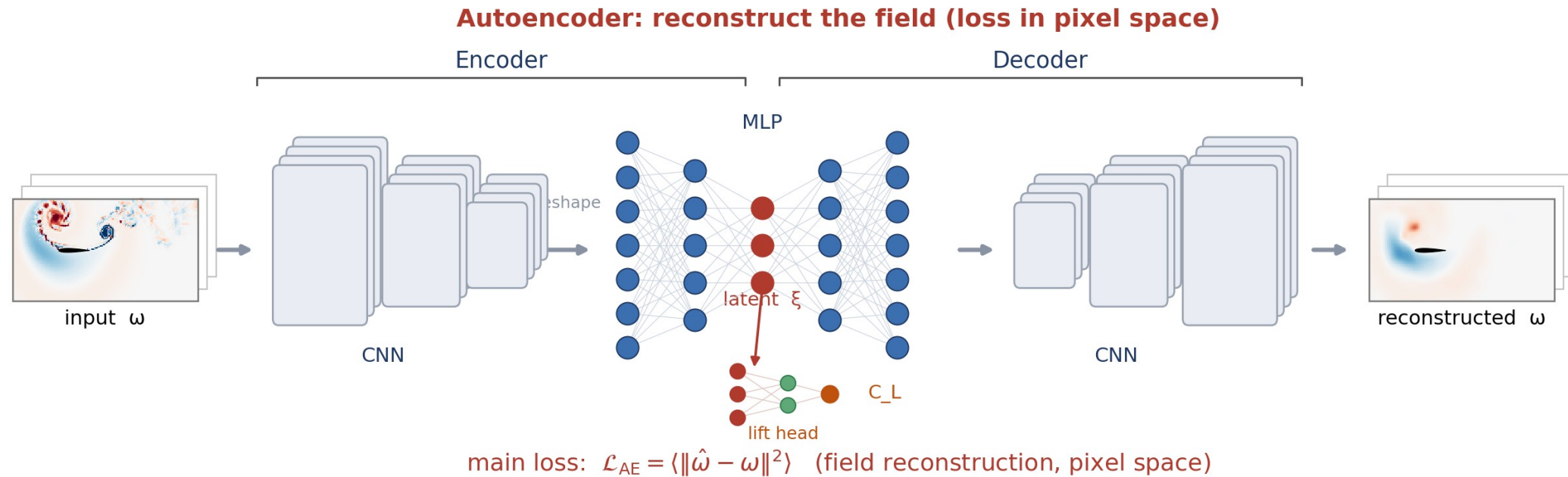
Our architecture for gust encounters



- The **encoder is unconditional**: a pure state map $\omega(t) \rightarrow z$; the gust never touches it
- The gust **$c = (G, D, Y)$ enters the predictor only** (AdaLN); the predictor **is** the latent dynamics model you plan or learn against
- A **visualisation decoder** is trained on the **frozen** encoder, never in the loss (figures only)
- Compared at matched latent dimension against a reconstructive AE (Fukami) and POD

Illustrated alternative: the autoencoder route

- Reconstructive route (Fukami-Taira lineage): a **CNN encoder**, an **MLP latent ξ** (which also feeds a lift head to C_L), and a **CNN decoder**; the loss compares the reconstructed field to the input in **pixel space**

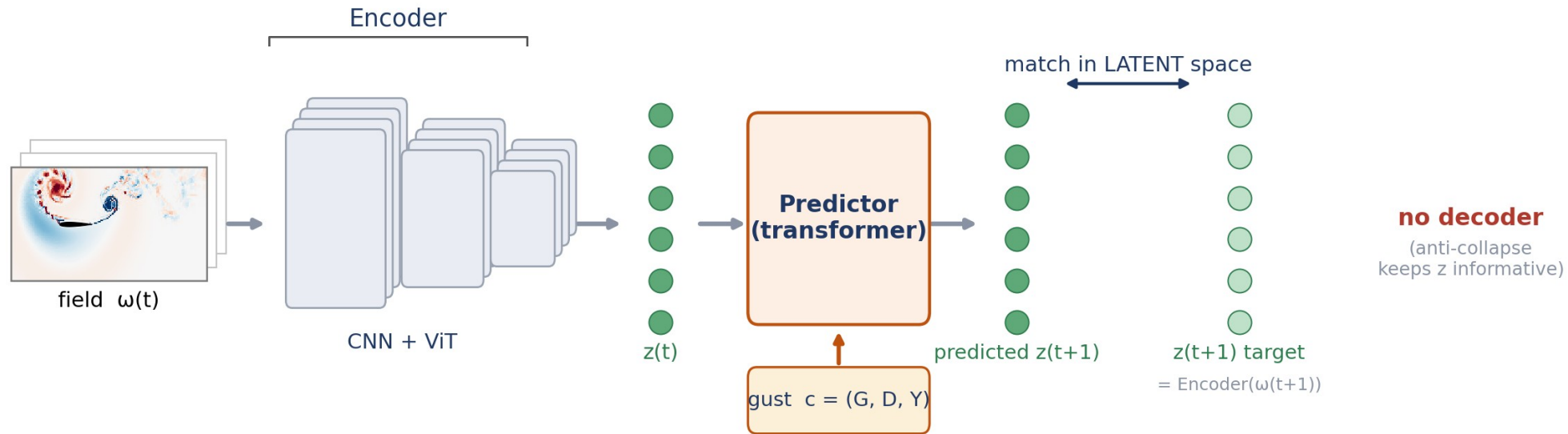


An autoencoder reconstructs the field; the latent is shaped by pixel reconstruction.

Illustrated alternative: the JEPA route

- Same encoder, but a **predictor** advances the latent and the loss lives in **latent space**, with **no decoder** (an anti-collapse term keeps z informative); the gust $c = (G, D, Y)$ enters the predictor only

JEPA: predict the next latent in latent space, no decoder

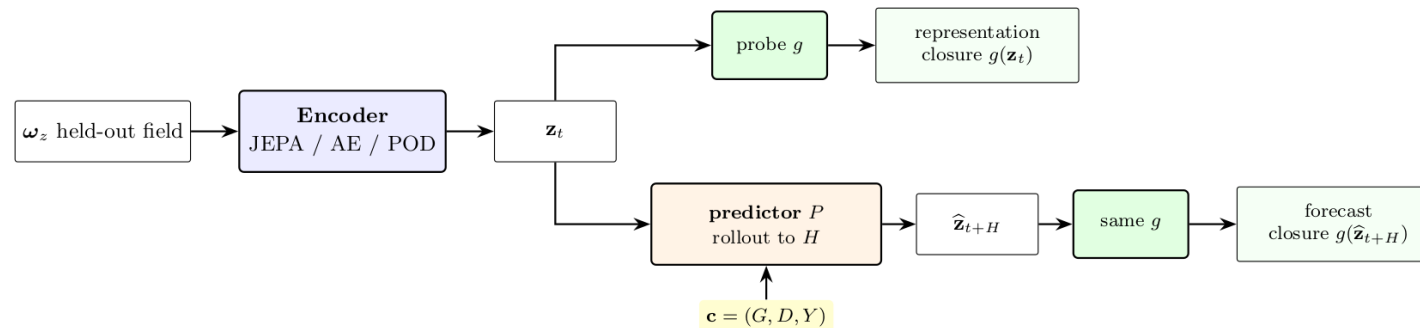


$$\text{main loss: } \mathcal{L}_{JEPA} = \langle \|\hat{z}_{t+1} - z_{t+1}\|^2 \rangle \text{ (latent space) } + \text{ anti-collapse (SIGReg)}$$

JEPA predicts the next latent in latent space, with no field reconstruction.

Forward-closure protocol

- **Forward closure:** roll the predictor recursively to $H = 16$ frames after impact, then probe each observable from the predicted latent
- **Matched protocol:** same predictor architecture and same probe family, **trained / fitted separately per latent**, so differences are the encoder's
- **Conditioning-only floor:** KRR on (G, D, Y) alone, the latent must beat it to prove the state, not the parameters, does the work
- Reported with bootstrap CIs, 3 encoder seeds, 5-fold probe CV; held-out test_b / test_c

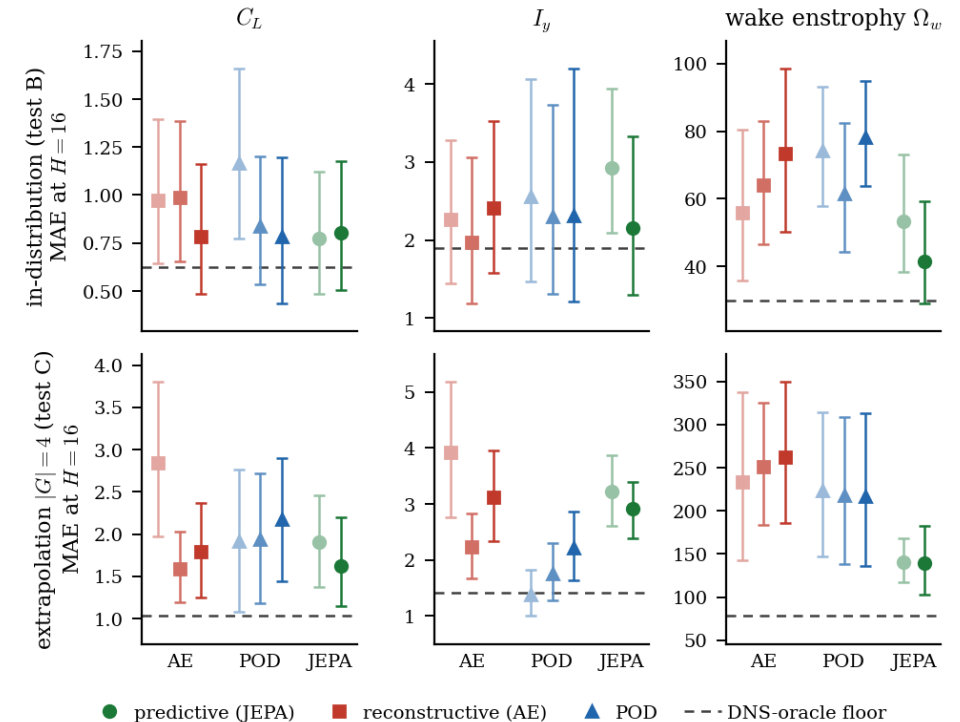


The predictor architecture, the conditioning $\mathbf{c}$, and the probe family g are identical across the three encoder families, each trained or fitted separately per family.

The same predictor and probe map every encoder family to the observables, in two modes.

Main result: forward closure

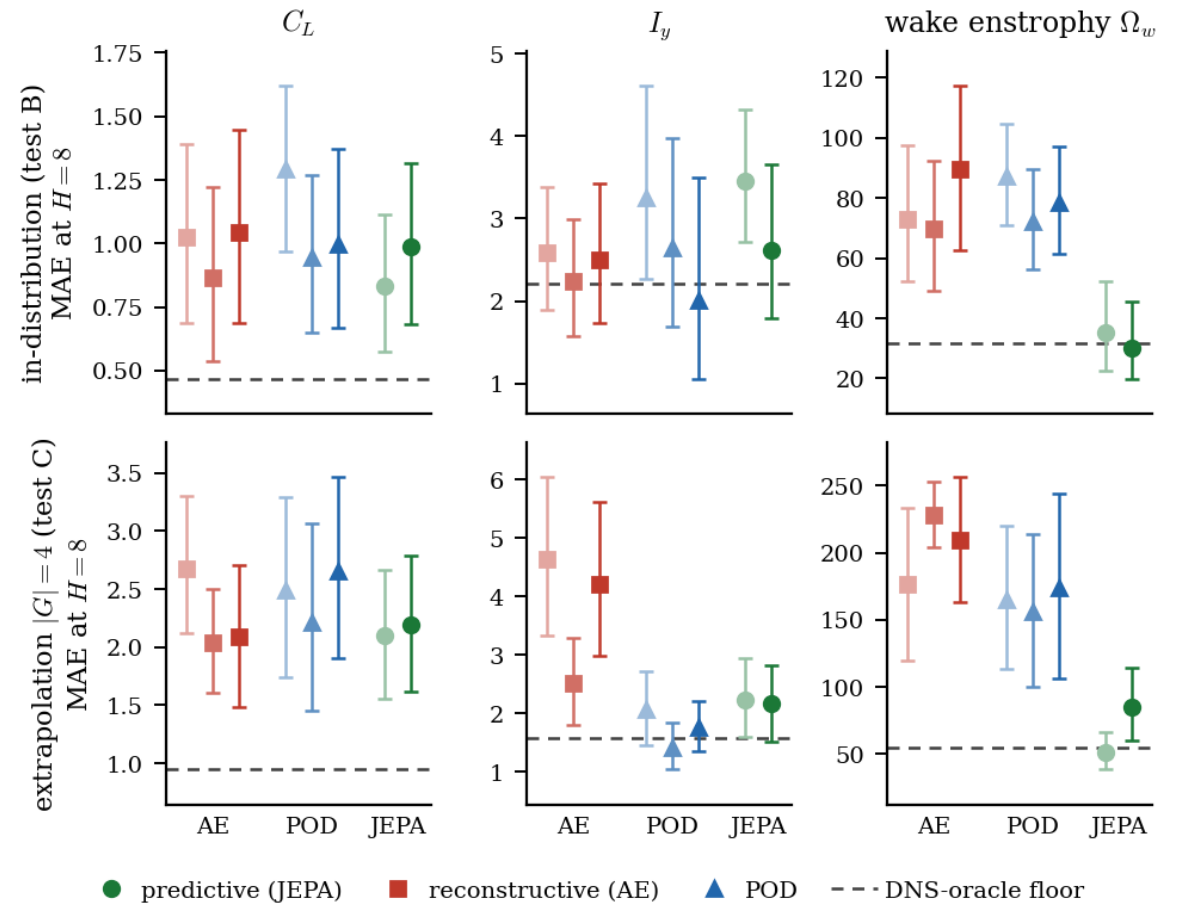
- Closure R^2 (rolled-out latent to observable, mean over six): **JEPA 0.84** • **Fukami 0.43** • **POD 0.56**
- **Wake enstrophy** is the discriminator: **0.93** vs 0.28 / 0.37; held-out MAE **2.4** to **3× lower**. POD competes only on the impulse I_y
- **Wake enstrophy** E_w : integral of ω_z^2 over the wake region (x/c in $[0.5, 4]$, $|y/c| \leq 1$)
- **Impulse** I_y : integral of $x \cdot \omega_z$ over the slice, a wake-transport diagnostic, not the impulse-theorem lift
- Each panel's MAE is in that observable's own units; R^2 is the cross-observable comparison



Held-out MAE at $H = 16$ by observable (test B top, test C bottom); each observable in its own units.

Forward closure at shorter horizons: $H = 8$

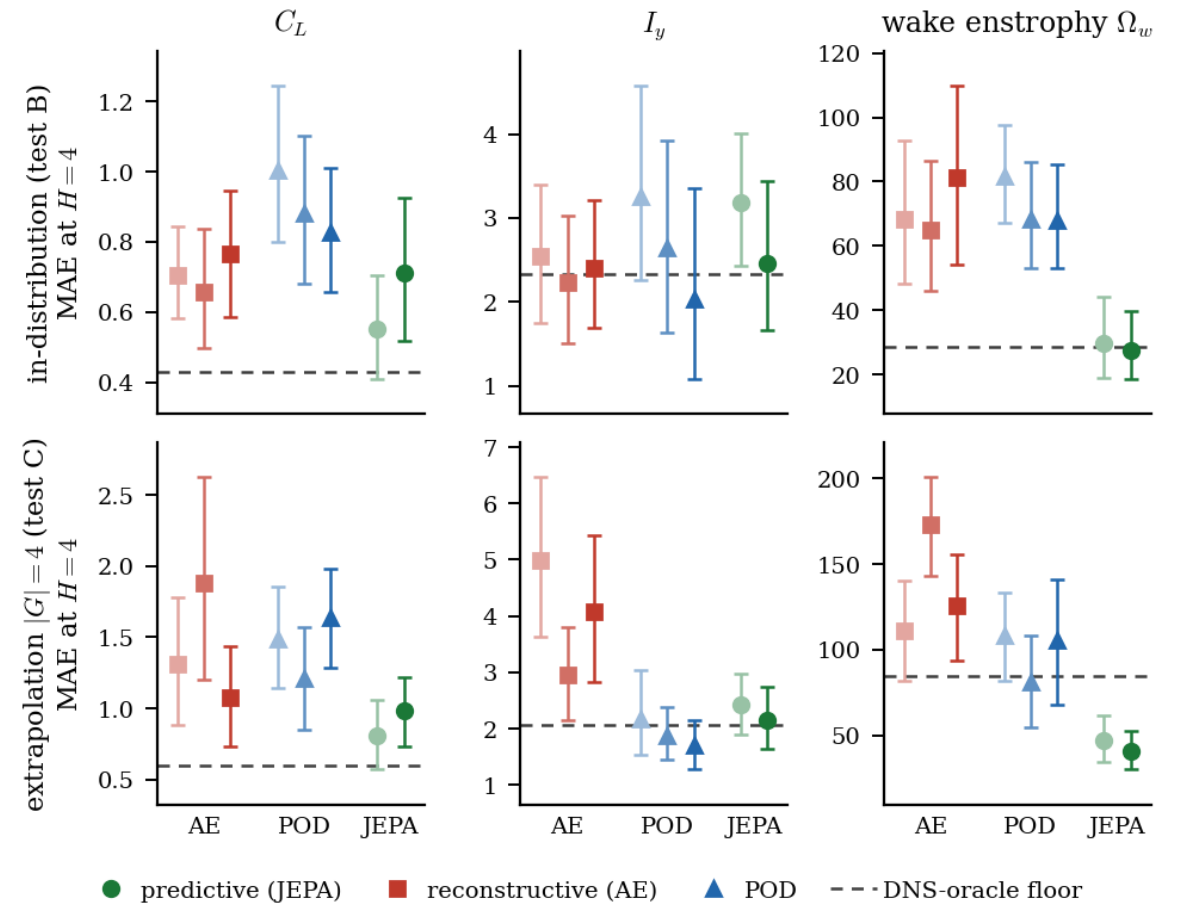
- The same comparison, rolled out to $H = 8$ instead of 16
- The family ordering is unchanged: the predictive latent is lowest on the wake, near the DNS-oracle floor
- Absolute errors are smaller at the shorter horizon: wake-
enstrophy MAE ≈ 30 for JEPA, vs ≈ 41 at $H = 16$
- Reconstructive and POD stay well above the floor on the wake



Held-out MAE at $H = 8$ by observable (test B top, test C bottom).

Forward closure at shorter horizons: $H = 4$

- Rolled out to $H = 4$, four frames after impact
- The predictive latent is still lowest on the wake enstrophy and closest to the oracle floor
- Errors shrink further: wake-enstrophy MAE ≈ 27 for JEPA; the wake stays the discriminating observable
- So the advantage holds at **every horizon we test**, not only $H = 16$

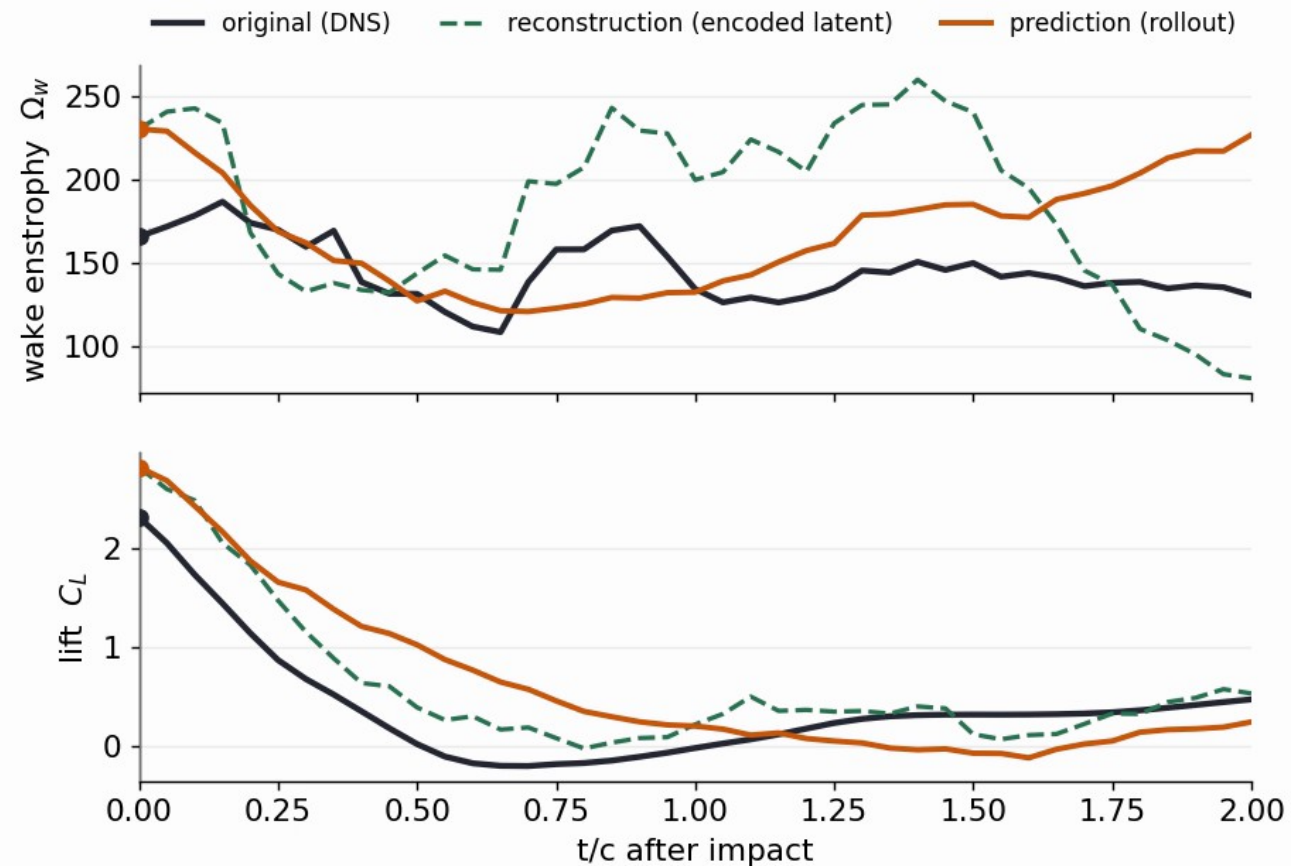


Held-out MAE at $H = 4$ by observable (test B top, test C bottom).

Forward closure in action: the rollout tracks the encounter

- The predictor **rolls the latent forward from impact**; each scalar is read off the rolled latent by a fixed linear probe
- **Wake enstrophy** and **lift** track the DNS truth (black) through impact and the lift dip
- Reconstruction (green) = encoded-latent **ceiling**; prediction (orange) = the **forecast**
- A representative **low-error** encounter ($G = -1.5$), not the hardest case

JEPA forecast, gust encounter ($G=-1.5$, $D=0.5$, $Y=-0.2$): $t/c = 0.00$

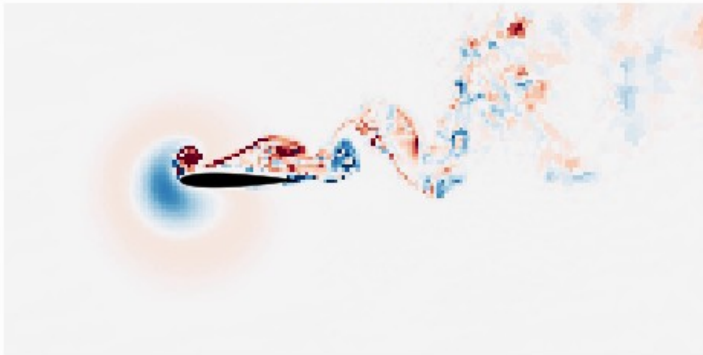


The same forecast in physical space

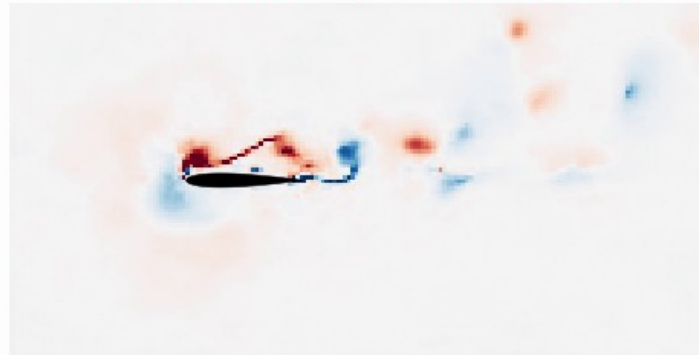
Decode the rolled latent every frame: the forecast as full vorticity fields, not just scalars.

Mid-plane vorticity, gust ($G=-1.5$, $D=0.5$, $Y=-0.2$): $t/c = 0.00$ after impact

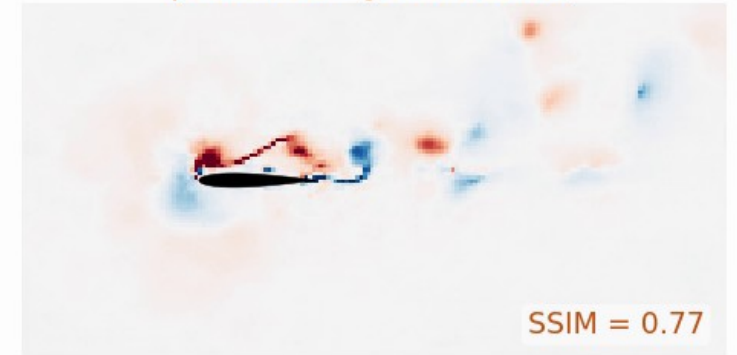
DNS truth



reconstruction (encoded latent)



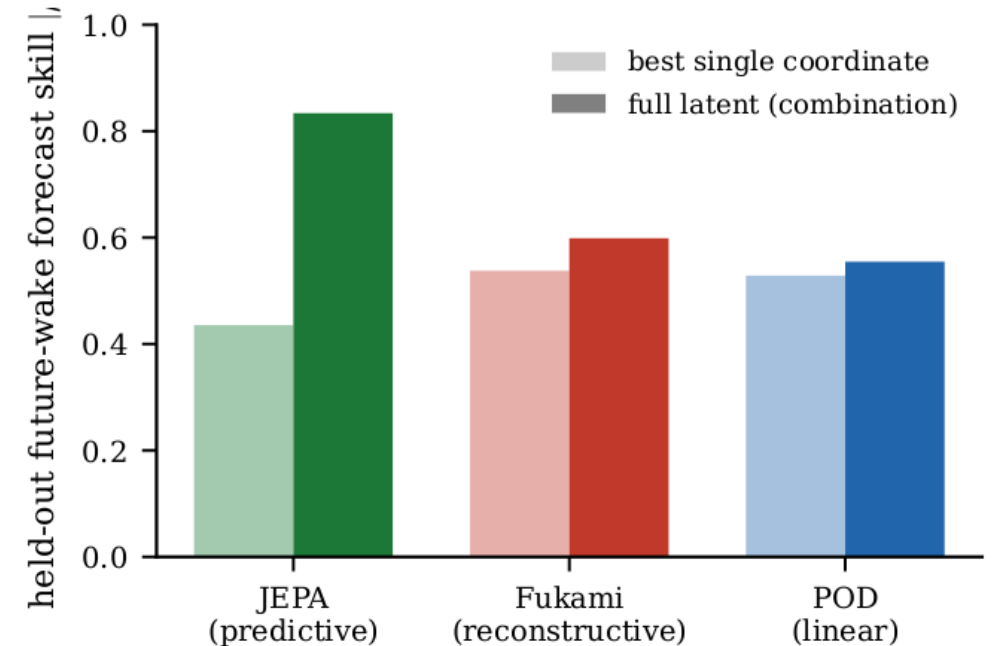
prediction (JEPA rollout)



DNS truth vs reconstruction (encoded latent) vs prediction (rolled latent), $G = -1.5$ encounter. SSIM(prediction) = 0.77 at impact, 0.53 at $H = 16$. The latent keeps the leading-edge vortex and shear layer; fine-scale wake is not retained at $d = 64$.

It is the wake, not the forces

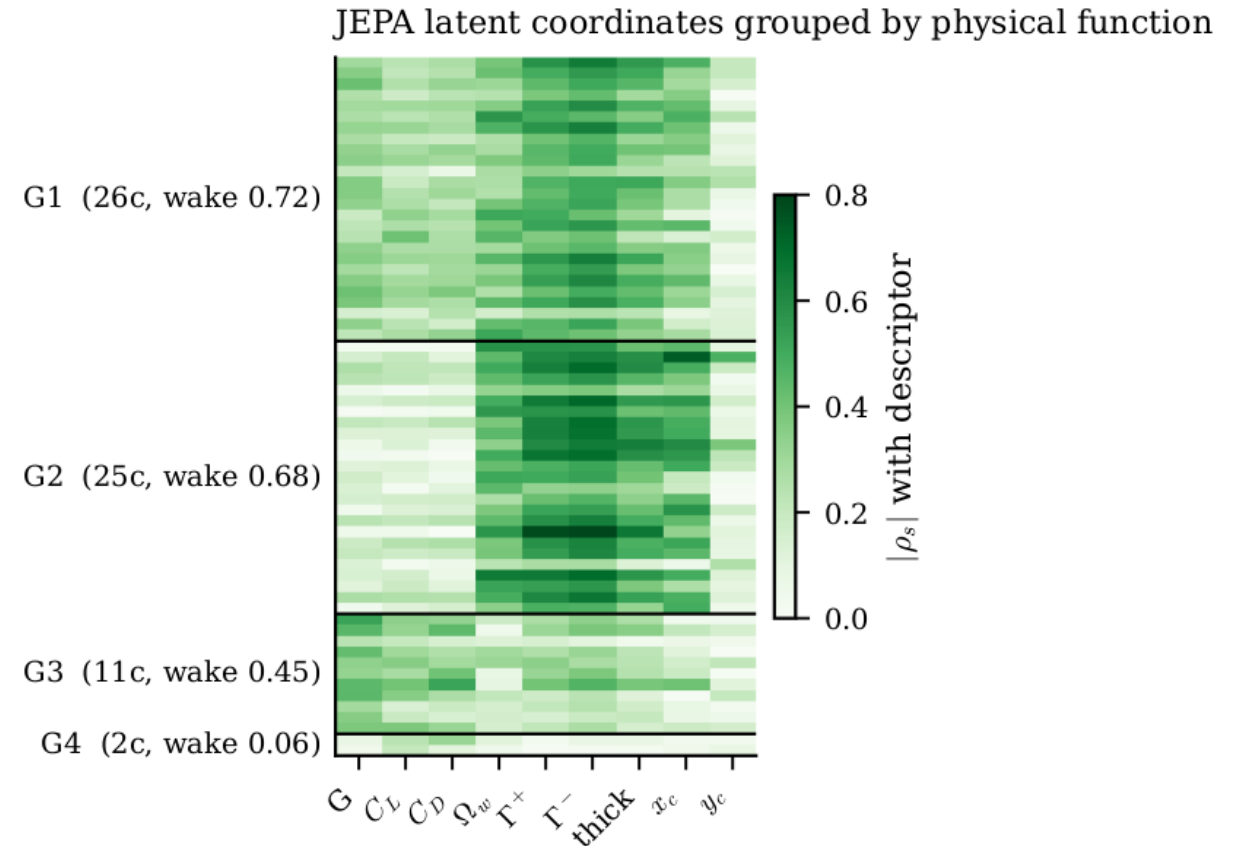
- **Forces** (C_L , C_D) are forecast **redundantly by every family**, many coordinates carry them
- Only the predictive latent carries a **distributed wake-forecast code** (rank corr 0.83; best single coordinate 0.44)
- Reconstructive / linear: best single coordinate $\approx$ the whole latent $\rightarrow$ **no collective wake structure**
- And it clears the **conditioning-only floor** $\rightarrow$ the latent, not the parameters, does the work



Wake forecast skill: full latent (dark) vs best single coordinate (light).

Latent coordinates group by physical function

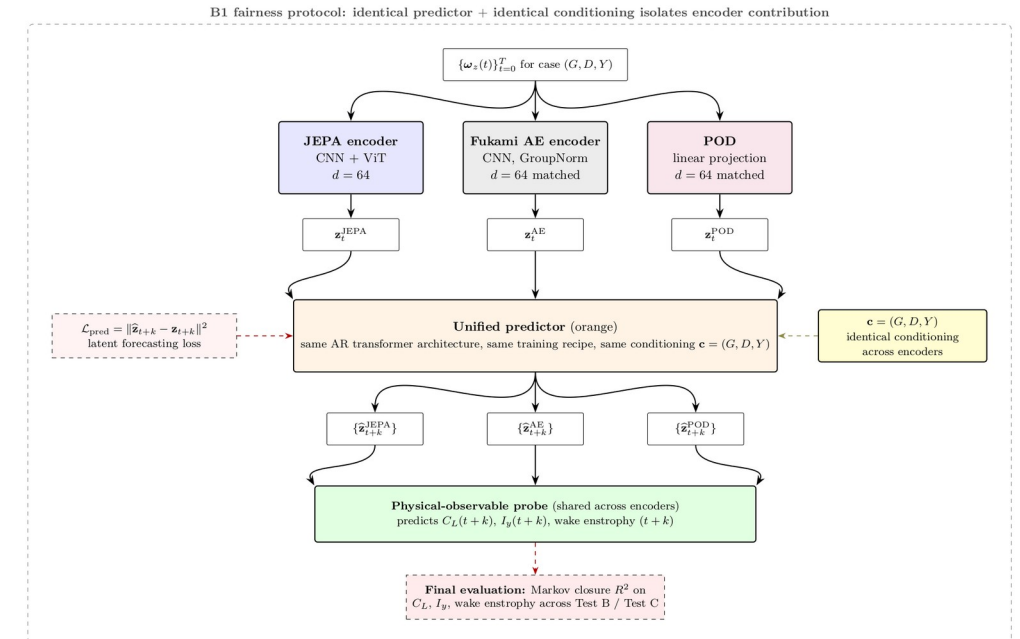
- Profile each of the 64 predictive-latent coordinates by its $|\text{Spearman}|$ correlation with nine descriptors (gust G , forces, wake enstrophy, circulations $\Gamma_{\pm}$, wake thickness, centroid)
- Clustering the profiles recovers **functional groups**: **~51 wake-vorticity** coordinates (G1, G2) and **11 gust-forcing** coordinates (G3); two near-silent
- Alone, the wake groups forecast the future wake at ≈ 0.7 , the forcing group at **0.45** (all 64 together: 0.83), the collective wake code seen earlier
- Read **descriptively**, not as a causal claim



Per-coordinate $|\rho|$ with each descriptor; rows grouped into functional clusters (G1-G4), labelled with coord count and held-out wake-forecast skill.

Controls: objective and supervision, not architecture

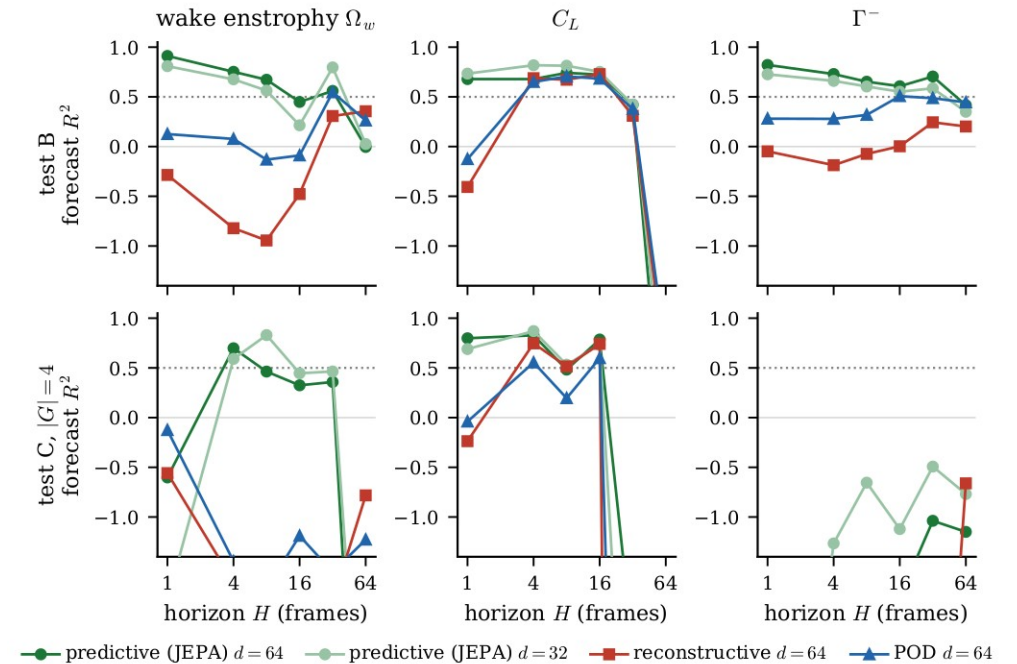
- **2 × 2:** objective {predictive, reconstructive} × architecture {CNN, CNN+ViT}, auxiliary heads matched
- Predictive beats reconstructive at **both** architectures (wake R^2 0.46 / 0.45 vs 0.16 / 0.29) → **not the ViT**
- Remove the wake head from the predictive model → wake closure **collapses below floor ($R^2 = -1.03$)**
- → the gain needs the **predictive objective and wake supervision together**



Objective × architecture controls, three seeds per cell.

Diagnostic 1: latent drift

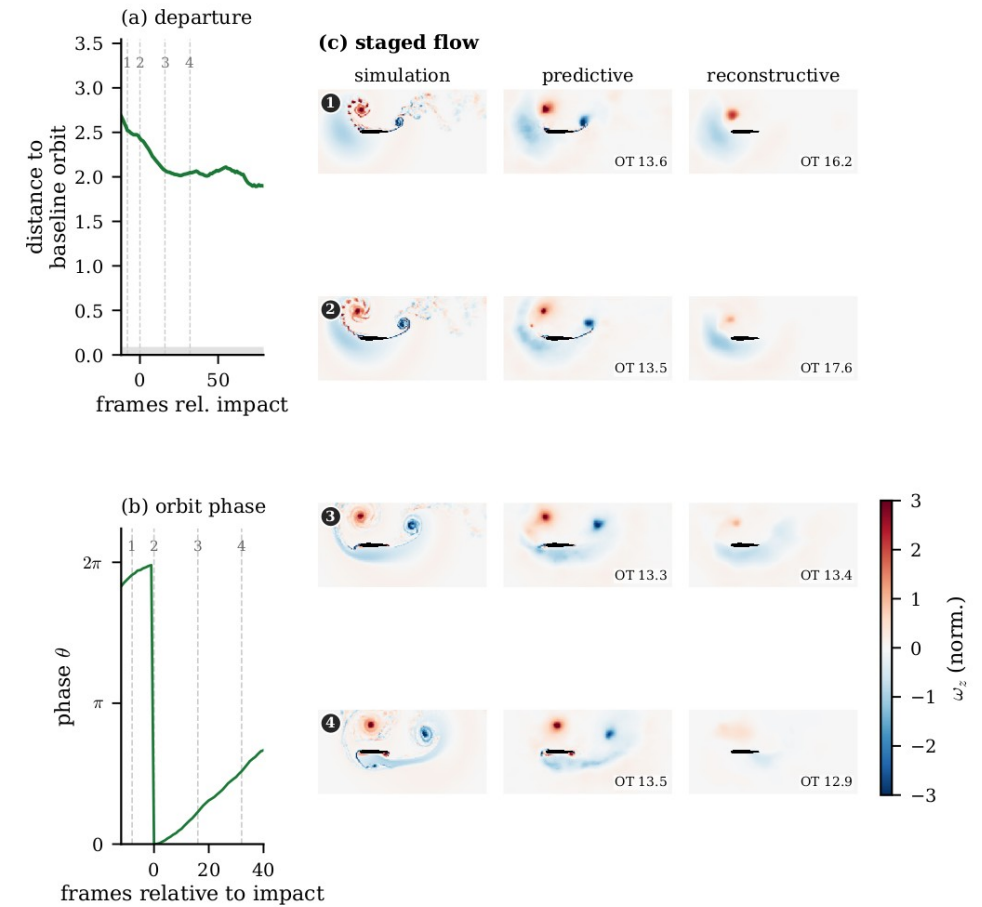
- **Why this question:** a planner / RL agent queries the model at states reached by its **own rollout**, one-step error is not enough
- **Mahalanobis distance:** covariance-aware distance from a reference cloud, $d = \sqrt{(z-\mu)' S^{-1} (z-\mu)}$; ≈ 1 is one standard-deviation unit
- We compare the **rolled-out** latent to the distribution of **DNS-encoded** latents (ratio = rollout / encoded)
- **Result:** the reconstructive rollout drifts **$\sim 10\times$ off-manifold (9.9)**; JEPA 0.85 and POD 0.81 stay inside
- The R^2 in the plot is **forward closure via a probe**: the predictor advances only the **latent**, and a fixed linear probe reads each observable off the rolled-out latent (no direct predictor of C_L or the wake)



Forward-closure R^2 vs rollout horizon (test_b): the predictor advances the latent; a linear probe reads each observable off it.

Diagnostic 2: topology of the encounter

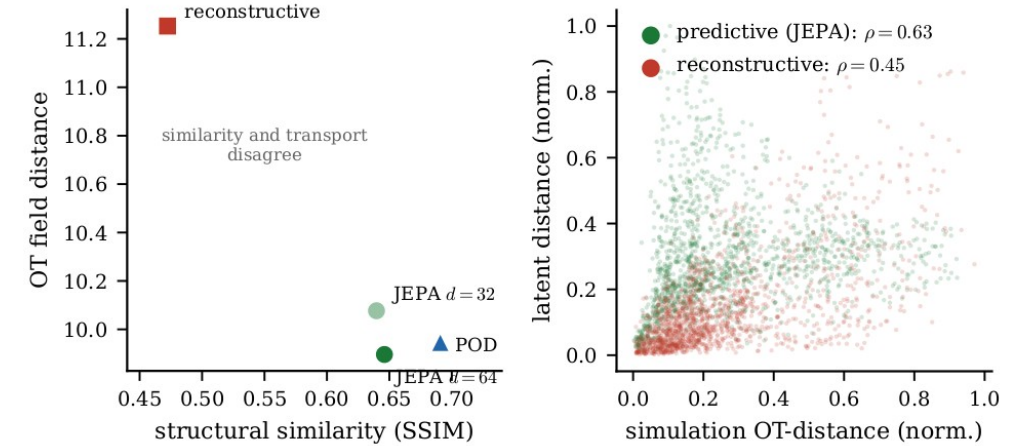
- **Why:** shedding and gust encounters are **cyclic**; a faithful state should preserve that cyclic structure
- **Persistent homology** (coordinate-free): counts loops (1-cycles) and how long they persist as a scale grows; **one long-lived cycle** = clean loop, many short-lived = fragmentation
- **Result:** the predictive latent has a **single persistent cycle**, the reconstructive latent **fragments**. This is the *topological* statement, **not** that the PCA curve visually closes
- Panels use **encoded latents + the decoder, no predictor**: in (PC1, PC2) the orbit **departs the baseline cycle and only partially returns** within the 120-frame window; the staged snapshots are reconstructions



(a, b) PCA of the encoded-latent trajectory; (c) staged decodes (reconstructions) with OT distances. The single-cycle is established by persistent homology; the orbit need not close in the window.

Diagnostic 3: transport geometry

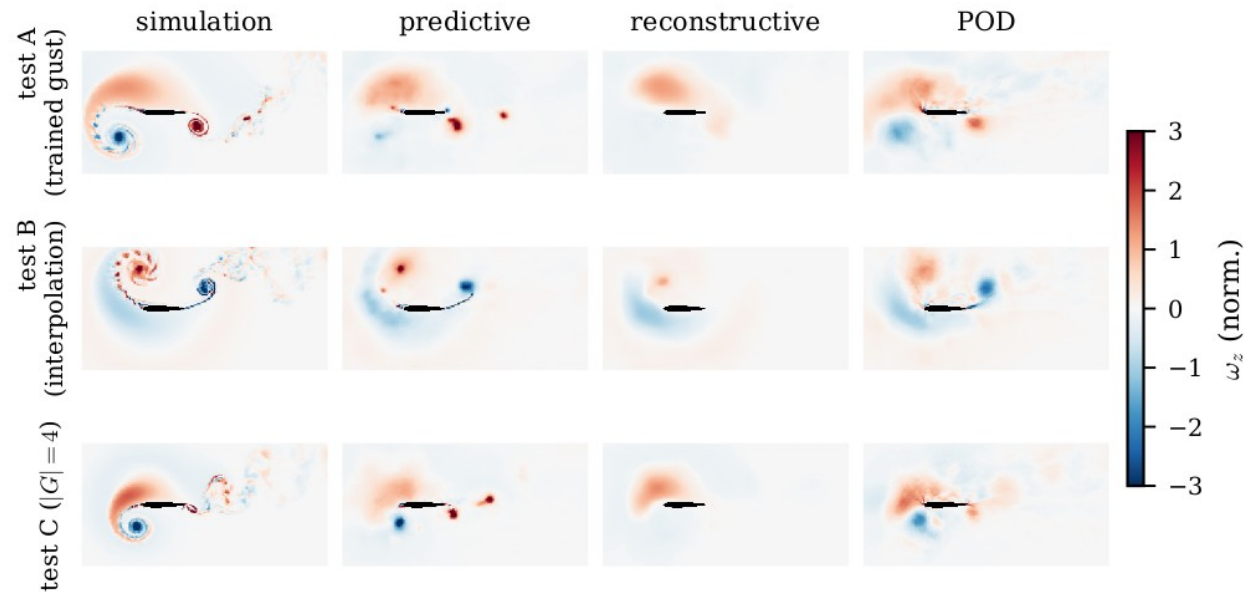
- **Why:** Euclidean / pixel distance ignores **where structures move**; control cares about advection of the LEV and shear layer
- **Optimal-transport (OT) distance:** the least cost to rearrange one vorticity field into another, sensitive to transport
- We Spearman-correlate the **latent** distance matrix with the **OT** distance matrix along each encounter
- **Result:** JEPA **0.63** vs Fukami 0.45, order-preservation, not an isometry; latent steps mean physically sensible field changes



Latent vs OT distance alignment (per-encounter Shepard).

Decoded reconstructions

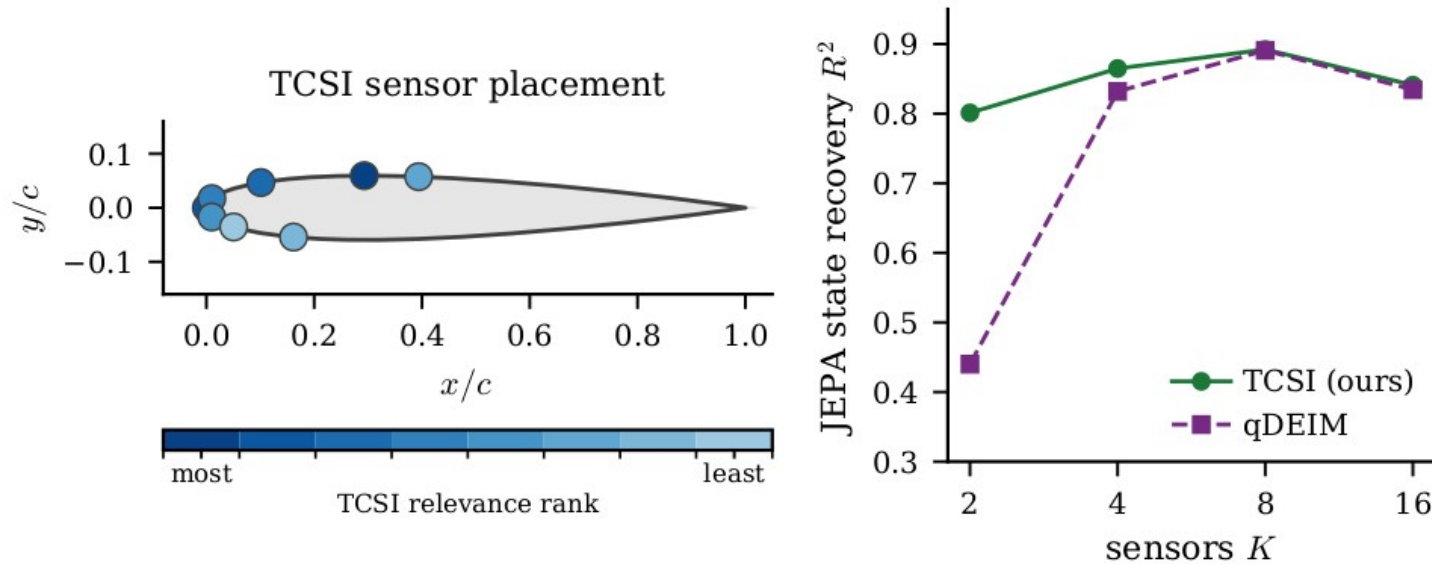
- Decoded fields: predictive vs reconstructive vs POD vs DNS
- The predictive decode is **blurrier pixel-wise** but preserves the **transported large-scale wake**; the reconstructive decode is sharp yet drifts under rollout



Held-out reconstructions; the predictive objective trades pixel sharpness for transported structure.

Sparse sensor placement

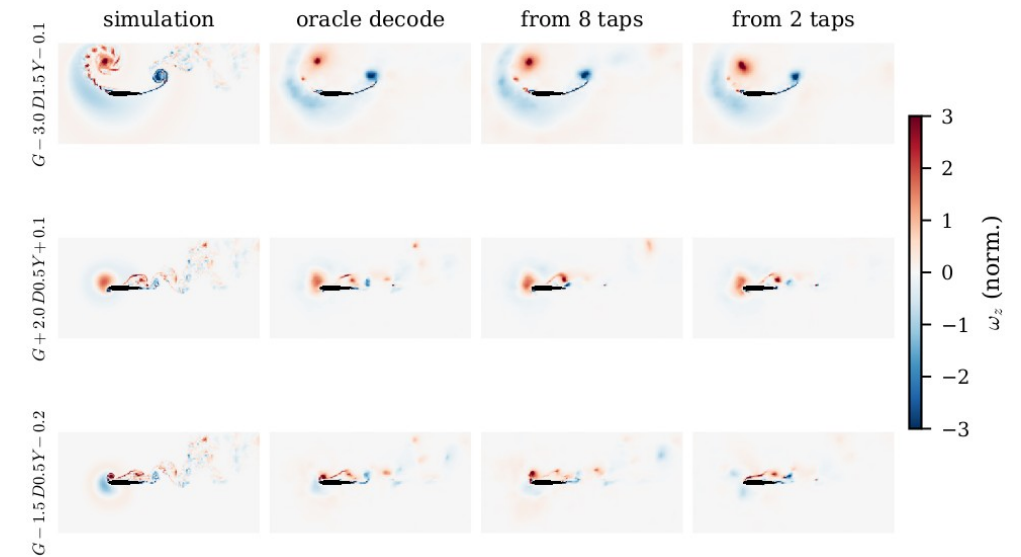
- Sparse wall-pressure sensors selected by **TCSI / qDEIM** vs uniform
- Pressure $\rightarrow$ latent map: KernelRidge(RBF) on the K -sensor $\times$ pre-impact-window vector
- $K = 2 / 4 / 8 / 16$; the LSTM/KRR comparison is 5-fold-CV selected to guard small-sample overfitting



Optimal sparse sensor placement on the airfoil surface.

Flow recovered from sparse wall pressure

- Estimate the **impact-frame predictive latent** from **K wall-pressure taps** (TCSI placement, kernel ridge), then **decode** it to a field
- **K = 8 taps** recover the leading-edge vortex and shear layer; **K = 2** coarsens but keeps the gross wake structure
- Benchmarked against the **oracle decode** (decode of the simulation-encoded latent) and the simulation
- So the predictive state is **reconstructible from a few wall sensors**, a deployment-relevant observability result

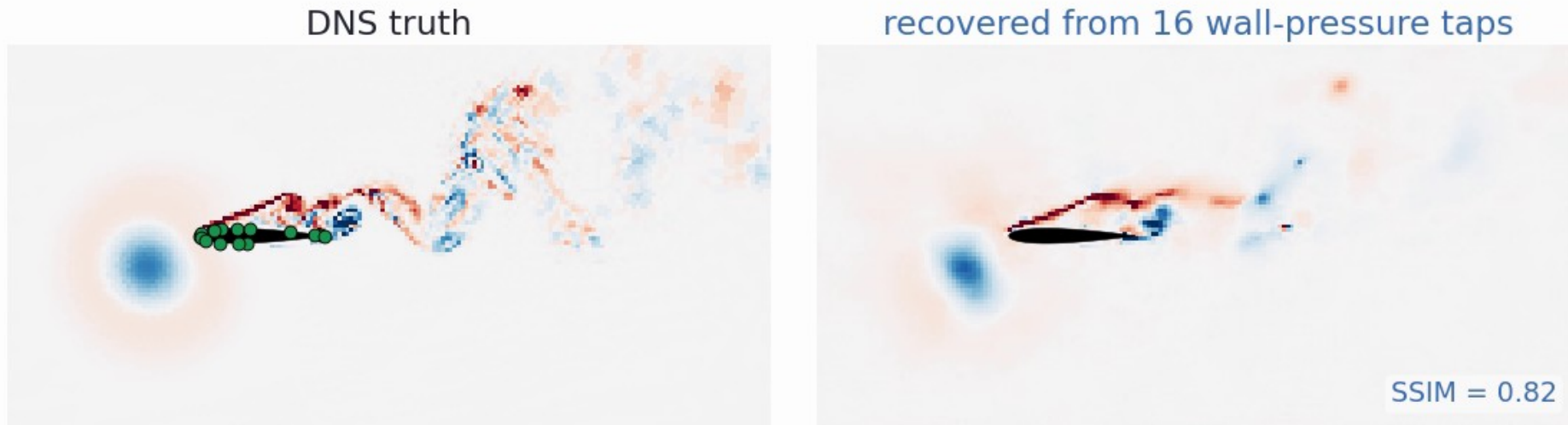


Flow recovered from sparse wall pressure: simulation, oracle decode, and decode of the latent estimated from $K = 8$ and $K = 2$ taps (held-out encounters).

Sparse-sensor state estimation in action

No predictor here: a per-frame map from 16 wall-pressure taps to the latent, decoded to a field.

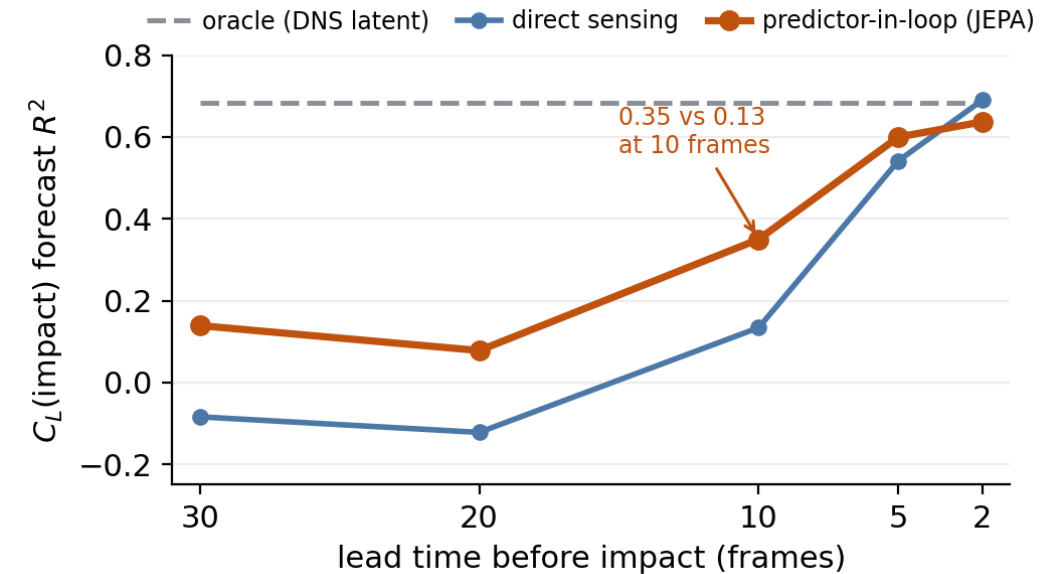
State estimate from 16 wall-pressure taps: $t/c = -0.40$ before impact



DNS truth (green dots mark the 16 TCSI taps) vs the field recovered from wall pressure alone, on a held-out encounter. Held-out-encounter latent R-squared = 0.72, SSIM ~ 0.5. Wall pressure fixes the near-body LEV and shear layer; the far wake is not surface-observable.

Control relevance: observable and forecastable

- The predictive state is **observable from sparse wall pressure**: (G, D) recoverable from the latent ($R^2 \approx 0.46 / 0.80$ on test_b)
- **Lead-time impact-C_L**: predictor-in-loop $R^2 = 0.35$ at 10 frames pre-impact, vs 0.13 from direct pressure sensing (oracle 0.68)
- The reconstructive latent's oracle is **negative**, a representation failure, not a probe failure
- A deployment-relevant **state estimate plus short-horizon forecast**: the ingredients a controller consumes



Impact-C_L inferred at each lead time before impact (test_b R^2): oracle = probe on the DNS-encoded latent; direct = pressure sensors to C_L ; predictor-in-loop = roll the latent to impact, then probe.

Takeaways

- A **predictive objective + wake supervision** yields a latent dynamics model that is:
 - **forward-closed** (wake R^2 0.75 representational / 0.84 mean), **on-manifold** under rollout, a **single cycle, transport-aligned**
 - **compact**, $d = 32 \approx d = 64$, participation ratio ≈ 1.7
 - **observable** from sparse wall pressure
- A **conditional forward-closure model**, a substrate for **model-based control and RL world-models**
- The discriminator throughout is the **wake**, which force-only and reconstruction-only states lose

Outlook

- Replace the gust conditioning channel with an **actuation channel**, the same forward-closure machinery then applies to control inputs
- **Closed-loop** model-based control / RL world-models built on the latent
- **3D observability** at strong gusts ($|G| = 4$ is a mid-plane-slice limit)
- The predictive-latent recipe carried to **other parametric flows**

THANKS

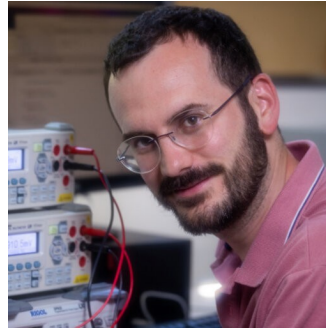
Acknowledgements

This work is a collaboration with:



Alberto Solera-Rico

INTA · UC3M



Arnau Miró

UPC · BSC



Oriol Lehmkuhl

BSC

Work produced with the support of a 2024 Leonardo Grant for Researchers and Cultural Creators, BBVA Foundation, project PREVENT (grant LEO24-2-15988). The Foundation takes no responsibility for the opinions, statements and contents of this project, which are entirely the responsibility of its authors.

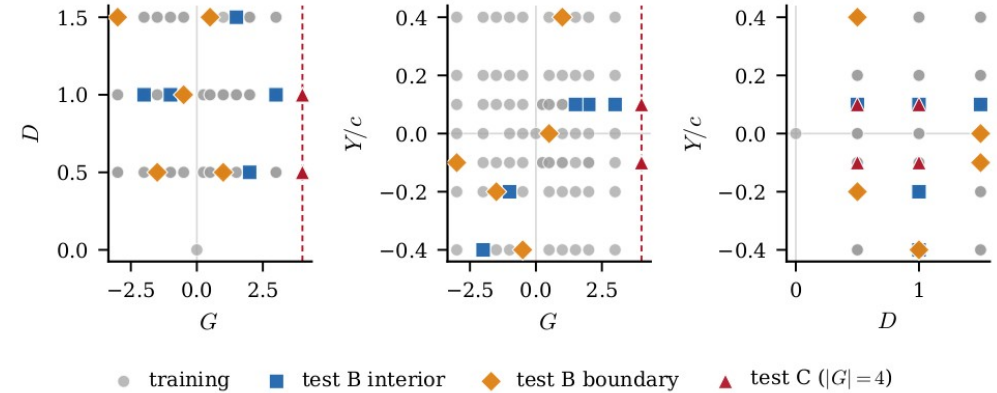


Backup slides

Supporting detail and ablations

Dataset and protocol

- Partition **v2** (locked, sha256-anchored): 84 cases
- **226** train encounters · **42** test_b · **24** test_c ($|G| = 4$)
- ω_z pipeline v1: spatial mask + p99.99 clip + 3σ scale (train_std 3.55), no mean shift (preserves vorticity antisymmetry)
- Reporting: bootstrap $n = 2000$ CIs · 3 encoder seeds · 5-fold probe CV



Training envelope in (G, D, Y) .

Matched-capacity: $d = 32$ vs $d = 64$

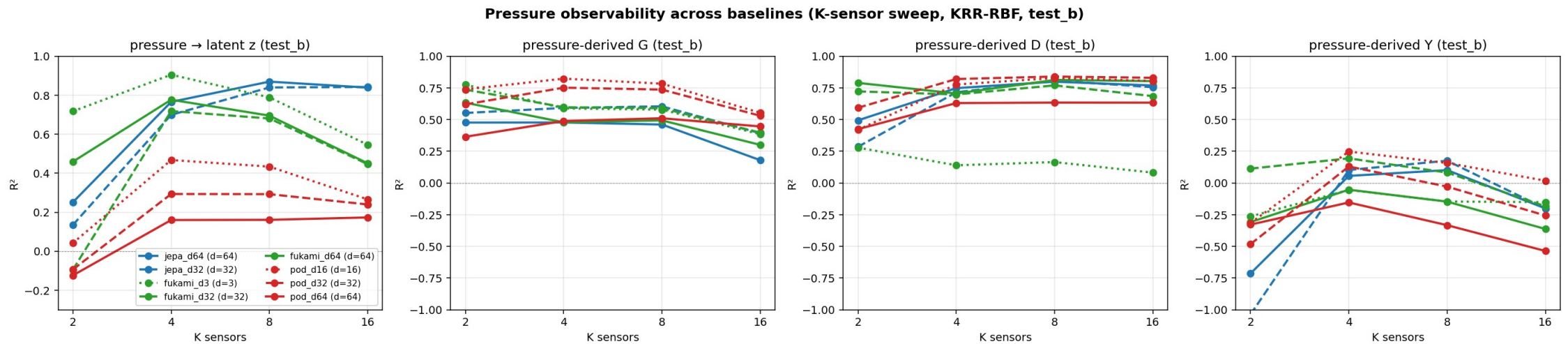
- Halving the latent leaves the representation and **every mechanism diagnostic** intact
 - representational wake R^2 0.74 ($d=32$) vs 0.75 ($d=64$); drift ratio 0.86 vs 0.85; OT 0.61 vs 0.63
- Cost is **in-distribution forecast sharpness**: $H = 16$ wake closure 0.45 $\rightarrow$ 0.21
- Participation ratio ≈ 1.7 , leading PC $\approx \frac{3}{4}$ of variance $\rightarrow$ the effective dimension is a handful

Seed variance

- Three encoder seeds per cell; wake-closure means reported $\pm$ standard deviation
- The reconstructive CNN+ViT cell has **large seed variance (± 0.27)**, consistent with the drift mechanism: without a forward-predictable geometry, closure is unstable across initialisations
- Predictive cells are tight (± 0.03 – 0.06)

Parameter observability from the latent

- $z \rightarrow (G, D, Y)$ probe R^2 from the rolled-out latent, $K = 8$ pre-impact sensors
- test_b: **G 0.46 • D 0.80 • Y 0.10**, gust impulse and depth recoverable; cross-stream offset Y is marginal
- test_c ($|G| = 4$): negative, the 3D observability boundary of a single mid-plane slice



Gust parameters implicit in the latent on held-out cases.

SSIM convention

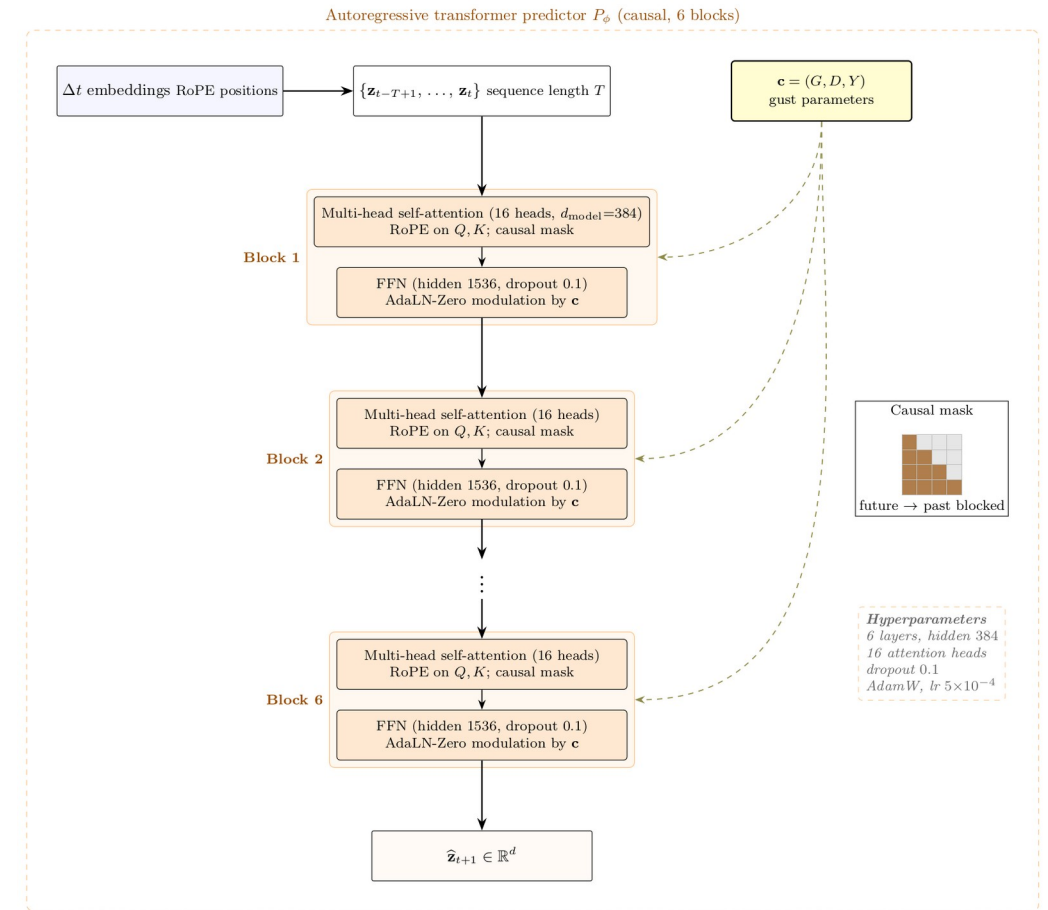
- Reconstruction SSIM uses the **Wang convention** ($K1 = 0.01$, $K2 = 0.03$) on pipeline-normalised ω
- Data range **$L = 2 \cdot \text{global p99.9}(|\text{target}|) \approx 8.31$** (split v2)
- Decoder test_a SSIM $\approx$ **0.71** at this convention

Conditioning-only floor

- KRR-RBF on (G, D, Y) → observable at impact; train / test_b / test_c R^2
 - on train, (G, D, Y) interpolates C_L and C_D well (226 points in 3-D)
- On **test_b** the floor **collapses** across observables; on **test_c** it is negative for 5 of 6
- → JEPA's generalisation is **not** explained by the conditioning alone

Model detail: encoder and predictor

- **Encoder** = a CNN stem (3 down-sampling stages) + a 6-layer Vision Transformer; the field becomes 288 patch tokens, and a learned [CLS] token is mapped by a small MLP (with BatchNorm) to the latent z ($d = 32 / 64$)
- **Predictor** = a 6-layer autoregressive transformer (hidden 384, 16 heads, dropout 0.1): it reads $z(t)$ and emits the next latent $z(t+1)$
- **Causal mask**: each step sees only past steps, never the future; **RoPE** encodes the frame ordering so the transformer knows what is earlier or later
- **AdaLN-Zero**: how the gust (G, D, Y) and shedding phase ϕ enter, they rescale the transformer's internal normalisation, initialised to do nothing so they perturb training gently
- **Training**: match the next latent (teacher forcing) + **scheduled sampling** (feed the model its own predictions over an 8-step rollout, so it tolerates its own errors) + **SIGReg** anti-collapse (keeps z informative without a decoder); AdamW, bf16, 80k steps



Phase-amplitude reading

- Phase-amplitude decomposition of the encounter cycle in the predictive latent
- Connects the predictor to the **sensitivity-function control** designed for these flows
- An actuation channel would enter where the gust does, the predictor conditioning

